

Tema 2.

Patrones léxicos y detección de falsos amigos

Control de calidad léxica en traducción automática biomédica

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Índice de contenidos

Título narrativo.....	1
Contexto profesional.....	1
Qué vas a aprender en esta unidad.....	1
Mapa conceptual de la unidad.....	2
1. Marco teórico.....	2
1.1. De la lectura humana a la vigilancia automática.....	2
1.2. Fundamentos de las expresiones regulares.....	3
1.3. Patrones léxicos en PLN.....	3
1.4. Falsos amigos: problema lingüístico y problema computacional.....	3
1.5. El dominio biomédico: precisión léxica y seguridad comunicativa.....	4
1.6. Textos bilingües y segmentos TO-TM.....	4
1.7. Expresiones regulares en Python.....	4
1.8. Glosarios como representación de conocimiento lingüístico.....	4
1.9. Informes de revisión, falsos positivos y falsos negativos.....	5
1.10. Integración en un pipeline de posesición.....	5
1.11. Ventajas, limitaciones y buenas prácticas.....	5
1.12. Conclusiones del marco teórico.....	5
2. Detección de patrones léxicos y falsos amigos con Python.....	6
2.1. Representar un texto multilínea en Python.....	6
2.2. El módulo re: expresiones regulares en Python.....	6
2.3. Cadenas raw: por qué escribimos r"\bactual\b".....	7
2.4. Límites de palabra: evitar capturas falsas.....	7
2.5. Buscar todas las coincidencias con re.findall().....	8
2.6. Localizar segmentos con split('\n').....	9
2.7. Limpiar líneas con strip().....	9
2.8. Comprobar presencia con re.search().....	9
2.9. Glosarios como diccionarios.....	10
2.10. Recorrer el glosario con un bucle for.....	11
2.11. Patrones dinámicos con f-strings raw.....	11
2.12. Primer tipo de informe: segmentos sospechosos.....	12

2.13. Segundo tipo de informe: error probable en TO y TM.....	12
2.14. Glosario de corrección: no solo avisar, también orientar.....	13
2.15. Qué ocurre con variantes morfológicas.....	14
2.16. Falsos positivos y falsos negativos.....	14
Errores frecuentes de aprendizaje.....	15
Síntesis final.....	15
Cierre narrativo.....	15
Referencias.....	16

Título narrativo

Control de calidad léxica en traducción automática biomédica

Contexto profesional

Imagina que trabajas como lingüista computacional en una empresa tecnológica que desarrolla un motor de traducción automática especializado en textos biomédicos. El equipo recibe documentos bilingües inglés-español procedentes de artículos científicos, registros clínicos, informes técnicos, patentes farmacéuticas y materiales de comunicación sanitaria.

El sistema de traducción automática produce versiones bastante fluidas, pero el equipo de calidad ha detectado un problema recurrente: algunos segmentos parecen correctos a primera vista y, sin embargo, contienen errores léxicos graves. Son errores que no siempre rompen la gramática de la frase, pero sí alteran el significado. En traducción biomédica, ese matiz importa. Una mala equivalencia puede afectar a la interpretación de un síntoma, a la descripción de un tratamiento, a la lectura de una contraindicación o a la confianza del destinatario.

Tu tarea consiste en diseñar una primera capa automática de vigilancia lingüística. No vas a sustituir a la persona revisora. Vas a construir una herramienta que le ayude a encontrar antes los segmentos sospechosos. El objetivo es automatizar lo repetitivo para que la revisión humana pueda concentrarse en lo que exige criterio experto.

En esta unidad aprenderás a usar expresiones regulares en Python para detectar patrones léxicos, localizar posibles falsos amigos, contar ocurrencias, generar informes por segmento y distinguir entre un término simplemente sospechoso y un falso amigo realmente cometido en la traducción.

La historia profesional de fondo es esta: el conocimiento lingüístico no se queda en una intuición. Se transforma en estructuras de datos, patrones de búsqueda y reglas de decisión.

Qué vas a aprender en esta unidad

Al terminar este manual, deberías poder explicar y aplicar los siguientes conocimientos:

1. Qué son las expresiones regulares y para qué sirven en procesamiento del lenguaje natural.
2. Cómo se usa el módulo `re` de Python.
3. Qué significa trabajar con textos bilingües en pares TO-TM.
4. Qué es un falso amigo y por qué puede ser crítico en traducción biomédica.
5. Cómo se construye un patrón para buscar una palabra exacta.
6. Qué función cumplen los límites de palabra `\b`.
7. Por qué conviene usar cadenas `raw`, como `r"\bactual\b"`.
8. Qué diferencia hay entre `re.findall()` y `re.search()`.
9. Cómo se usa `re.IGNORECASE` para ignorar mayúsculas y minúsculas.
10. Cómo se organiza un glosario de falsos amigos mediante diccionarios.
11. Cómo se recorre un diccionario con un bucle `for`.
12. Cómo se construyen patrones dinámicos con f-strings `raw`.
13. Cómo se divide un texto multilínea con `.split("\n")`.
14. Cómo se filtran líneas vacías con `.strip()`.
15. Cómo se genera un informe automático de segmentos sospechosos.
16. Cómo se reduce el ruido exigiendo una coincidencia doble en texto origen y texto meta.

17. Qué limitaciones tiene una solución basada en expresiones regulares.
18. Cómo podría integrarse esta técnica en un flujo profesional de posesición y control de calidad.

Mapa conceptual de la unidad

Concepto	En Python	En PLN y traducción
Expresión regular	Patrón formal de búsqueda	Permite localizar formas lingüísticas recurrentes
Módulo re	Biblioteca estándar de Python	Herramienta básica para búsqueda y validación textual
Texto multilínea	String entre comillas triples	Simula segmentos bilingües de revisión
TO-TM	Dos partes de un segmento	Texto origen y texto meta
Falso amigo	Par léxico problemático	Error de transferencia entre lenguas
\b	Límite de palabra	Evita capturar palabras dentro de otras
re.findall()	Lista de coincidencias	Recuento de términos sospechosos
re.search()	Primera coincidencia o None	Filtro para decidir si mostrar un segmento
Diccionario	Pares clave-valor	Glosario de riesgo o glosario de corrección
Bucle for	Repetición sobre elementos	Revisión sistemática de muchos términos
and	Condición doble	Detecta coincidencia en origen y traducción
Informe automático	Salida estructurada con <code>print()</code>	Apoyo a la revisión humana

1. Marco teórico

Esta unidad se apoya en una idea central: algunos errores léxicos de la traducción automática pueden parecer formalmente correctos, pero ser semánticamente peligrosos. Para detectarlos de forma sistemática, el conocimiento lingüístico se convierte en patrones explícitos, glosarios y reglas de decisión. El marco teórico organiza esa transición desde la revisión humana hasta una primera capa automática de vigilancia léxica.

1.1. De la lectura humana a la vigilancia automática

Una persona revisora puede detectar un falso amigo leyendo un segmento completo, pero cuando el corpus contiene cientos o miles de segmentos la lectura manual íntegra se vuelve lenta y propensa a omisiones. La programación permite transformar parte de ese conocimiento experto en un procedimiento repetible: buscar formas sospechosas, localizar segmentos, comprobar si aparece una traducción problemática y producir una alerta para revisión.

Esta automatización no sustituye la posesición ni la revisión terminológica. Su función es priorizar. En lugar de preguntar al sistema si una traducción es correcta, se le pide algo más limitado y verificable:

localizar patrones de riesgo. Esa diferencia es importante porque evita atribuir al código una comprensión semántica que no tiene.

1.2. Fundamentos de las expresiones regulares

Las expresiones regulares tienen su base en la teoría de lenguajes formales y en la representación de conjuntos de cadenas mediante operadores como concatenación, alternativa y repetición (Kleene, 1956). En términos prácticos, una expresión regular es un patrón formal que permite buscar secuencias de caracteres en un texto.

En PLN, las expresiones regulares resultan útiles para tareas de limpieza, validación, extracción y detección de patrones recurrentes. Su fuerza está en la precisión formal: si el patrón está bien definido, el sistema puede aplicarlo de manera rápida y consistente sobre muchos textos. Su límite está en que la coincidencia formal no equivale a interpretación lingüística.

Elemento	Función	Ejemplo en la unidad
Patrón léxico	Describe una forma o secuencia que se quiere localizar	<code>r"\bactual\b"</code>
Límite de palabra	Evita capturar fragmentos dentro de otras palabras	<code>\b</code>
Búsqueda insensible a mayúsculas	Agrupar <i>Actual</i> , <i>ACTUAL</i> y <i>actual</i>	<code>re.IGNORECASE</code>
Coincidencia doble	Exige evidencia en origen y meta	<i>actually</i> + <i>actualmente</i>

1.3. Patrones léxicos en PLN

Un patrón léxico captura unidades lingüísticas de interés mediante una forma formalizable: una palabra completa, una familia de variantes, una secuencia recurrente o una relación entre dos formas. En esta unidad, el patrón más importante es la palabra completa delimitada por fronteras, porque se busca detectar falsos amigos sin confundirlos con fragmentos de palabras más largas.

Los patrones léxicos se sitúan en una zona intermedia entre la lingüística y la programación. La lingüística decide qué forma es relevante y por qué; la programación define cómo localizarla de manera automática. Por ejemplo, saber que *actual* y *actually* no equivalen exige conocimiento contrastivo. Expresar esa alerta como `r"\bactual\b"` o `r"\bactually\b"` exige conocimiento técnico.

1.4. Falsos amigos: problema lingüístico y problema computacional

Un falso amigo es una forma de una lengua que se parece a una forma de otra lengua, pero cuyo significado no coincide, o no coincide en el contexto de uso. La semejanza formal aumenta el riesgo de transferencia literal. En traducción inglés-español, pares como *actually/actualmente*, *library/librería* o *condition/condición* pueden activar errores si se traducen sin atender al contexto.

Desde el punto de vista computacional, el falso amigo se puede representar como una relación entre dos señales: una forma sospechosa en el texto origen y una forma problemática en el texto meta. Detectar solo la forma inglesa indica riesgo; detectar además la traducción literal problemática permite hablar de error probable. Esta distinción entre riesgo y evidencia es una de las decisiones teóricas más importantes de la unidad (Chamizo Domínguez, 2008).

Nivel de detección	Qué busca	Interpretación
Riesgo	Aparece la forma origen sospechosa	Conviene revisar el segmento
Error probable	Aparecen forma origen y traducción problemática	Hay más evidencia para priorizar la revisión
Corrección orientativa	Se añade una propuesta terminológica	El sistema ayuda, pero no decide por la revisora

1.5. El dominio biomédico: precisión léxica y seguridad comunicativa

En traducción biomédica, los falsos amigos son especialmente delicados porque pueden afectar a síntomas, tratamientos, contraindicaciones, instrucciones clínicas o descripciones de estado. Una traducción que suena natural puede ser inadecuada si altera una relación terminológica o si usa una palabra general donde se requiere una equivalencia especializada.

Por eso, una capa de vigilancia léxica puede funcionar como una medida inicial de seguridad lingüística. No garantiza la calidad final, pero ayuda a señalar errores repetitivos que pueden pasar desapercibidos cuando el sistema de traducción automática produce una frase fluida.

1.6. Textos bilingües y segmentos TO-TM

En un flujo profesional de traducción y posesición, el texto se organiza a menudo en segmentos: unidades alineadas que contienen un texto origen y un texto meta. En esta unidad se usa la distinción TO-TM: TO es el texto origen, en inglés; TM es el texto meta, en español.

Separar TO y TM mejora la precisión de la detección, porque permite buscar *actually* en el origen y actualmente en la traducción. En un prototipo didáctico, ambos pueden aparecer en una misma línea bilingüe; en una herramienta profesional, convendría separarlos en campos distintos, por ejemplo en una tabla, archivo XLIFF, TMX o base de datos.

1.7. Expresiones regulares en Python

Python incorpora las expresiones regulares mediante el módulo `re`. Funciones como `re.findall()` y `re.search()` permiten aplicar patrones a cadenas de texto. `re.findall()` devuelve todas las coincidencias y resulta útil para contar; `re.search()` se detiene en la primera coincidencia y resulta útil para filtrar segmentos (Python Software Foundation, 2026).

Los patrones suelen escribirse como cadenas raw, por ejemplo `r"\bactual\b"`, porque las expresiones regulares usan barras invertidas y Python también les asigna significados especiales. La `r` inicial evita que Python interprete esas barras antes de que lleguen al motor regex. Esta buena práctica reduce errores invisibles al escribir patrones.

1.8. Glosarios como representación de conocimiento lingüístico

Un glosario de falsos amigos no es solo una lista de palabras: es una representación de conocimiento contrastivo. En Python puede organizarse como un diccionario donde la clave sea la forma inglesa sospechosa y el valor sea la traducción literal problemática. Un segundo diccionario puede almacenar alternativas recomendadas.

Este diseño permite pasar de búsquedas aisladas a un sistema escalable. Si se añade una nueva entrada al glosario, el bucle de detección no cambia. La calidad del sistema depende entonces tanto del código como de la calidad lingüística del glosario: selección de términos, contexto, dominio de uso, ejemplos y notas terminológicas.

1.9. Informes de revisión, falsos positivos y falsos negativos

Un informe automático debe ayudar a decidir. El primer nivel de informe puede listar segmentos sospechosos; el segundo puede mostrar errores probables cuando coinciden forma origen y traducción problemática; un tercer nivel puede añadir una propuesta de corrección. Cuanto más estructurado sea el informe, más útil será para la revisión humana.

Todo sistema de detección debe asumir falsos positivos y falsos negativos. Un falso positivo aparece cuando el sistema marca un caso que no es error; un falso negativo aparece cuando no marca un error real. En control de calidad, el equilibrio entre cobertura y precisión depende del objetivo. Si se quiere no dejar escapar errores, se aceptará más ruido. Si se quiere priorizar revisión, se exigirá más evidencia.

1.10. Integración en un pipeline de posesición

En un flujo profesional, las expresiones regulares no actúan de forma aislada. Pueden integrarse en un pipeline de control de calidad: recibir segmentos bilingües desde una herramienta TAO, normalizar codificación y saltos de línea, separar TO y TM, aplicar glosarios, generar alertas, revisar humanamente, validar terminología y actualizar el glosario.

Esta integración muestra que una solución sencilla puede tener valor profesional si está bien situada. Python no entrega una traducción final; entrega una lista priorizada de lugares donde mirar. Esa lista puede ahorrar tiempo, mejorar trazabilidad y reforzar la seguridad comunicativa en dominios sensibles.

1.11. Ventajas, limitaciones y buenas prácticas

Las expresiones regulares ofrecen precisión, velocidad y transparencia. Son adecuadas para patrones concretos, errores recurrentes y comprobaciones de superficie. Además, son explicables: una revisora puede entender qué busca el patrón y por qué se activa una alerta.

Pero también tienen límites claros. No entienden significado, no resuelven contexto completo, dependen del patrón y dependen del glosario. Si un falso amigo no está incluido, no se detectará. Si el patrón es demasiado estrecho, perderá variantes. Si es demasiado amplio, generará ruido.

Por eso conviene empezar con patrones simples, probarlos con ejemplos controlados, usar límites de palabra, ignorar mayúsculas cuando proceda, separar origen y meta, documentar el glosario, guardar ejemplos de falsos positivos y falsos negativos, y mantener siempre una fase de revisión humana.

1.12. Conclusiones del marco teórico

La unidad demuestra cómo una pregunta lingüística puede convertirse en un procedimiento computacional. El punto de partida es un riesgo de traducción: falsos amigos en textos biomédicos. El punto de llegada es una herramienta inicial de control de calidad que localiza patrones, organiza glosarios y genera informes de revisión.

La clave conceptual es que las expresiones regulares no sustituyen el criterio traductológico. Lo amplifican cuando se usan para tareas concretas, delimitadas y documentadas. Una buena alerta automática nace de una buena hipótesis lingüística; una buena hipótesis necesita después ser revisada en contexto.

2. Detección de patrones léxicos y falsos amigos con Python

2.1. Representar un texto multilínea en Python

Para trabajar con varios segmentos, Python permite guardar un bloque de texto completo en una variable. Se usan comillas triples:

```
texto = """
The patient is actually very sensible to the drug. El paciente es actualmente muy
sensible al medicamento.
Eventually, the physician decided to assist the patient. Eventualmente, el médico
decidió asistir al paciente.
The current treatment has no effect. El tratamiento actual no tiene efecto.
This is an actual problem that needs attention. Este es un problema actual que requiere
atención.
Eventually, the patient decided to look for a second opinion. Al final, el paciente
decidió buscar una segunda opinión.
"""
```

Este objeto es un string. La diferencia respecto a una frase corta es que contiene saltos de línea. Los saltos de línea forman parte del texto y pueden usarse después para separar segmentos.

La variable `texto` contiene todo el material que se va a revisar. Esta idea es importante: antes de analizar un corpus, hay que representarlo de alguna forma en memoria. En un prototipo inicial puede ser un string. En proyectos profesionales puede ser un archivo `.txt`, una hoja de cálculo, un archivo `.tmx`, un `.xliff` o una base de datos.

2.2. El módulo `re`: expresiones regulares en Python

Python tiene muchas funciones incorporadas, pero las expresiones regulares se gestionan mediante el módulo `re`.

```
import re
```

Un módulo es un conjunto de herramientas ya preparadas. `re` pertenece a la biblioteca estándar de Python, así que no hace falta instalarlo.

Una expresión regular es un patrón de búsqueda. Puede ser muy simple, como una palabra:

```
"actual"
```

O puede incluir instrucciones especiales:

```
r"\bactual\b"
```

Las expresiones regulares permiten buscar más allá de una coincidencia literal simple. Pueden expresar ideas como:

- *Busca esta palabra solo si aparece completa.*
- *Busca esta forma aunque esté en mayúsculas.*
- *Busca uno o más dígitos.*
- *Busca una secuencia que tenga este formato.*
- *Busca una palabra al principio o al final de una línea.*

En PLN son especialmente útiles para tareas de limpieza, extracción, validación y detección de patrones recurrentes.

2.3. Cadenas raw: por qué escribimos `r"\bactual\b"`

En Python, una barra invertida `\` puede tener significados especiales. Por ejemplo, `\n` representa un salto de línea. En expresiones regulares también se usan barras invertidas para símbolos especiales, como `\b`.

Para evitar que Python interprete esas barras antes de que lleguen al motor de expresiones regulares, se usa una cadena raw:

```
r"\bactual\b"
```

La `r` inicial significa *raw*, es decir, «cruda». Indica a Python que trate las barras invertidas de forma literal.

Sin la `r`, algunos patrones pueden funcionar, pero otros pueden comportarse de forma inesperada. Por eso, como buena práctica, los patrones regex suelen escribirse como cadenas raw.

La lectura del patrón es:

```
\b    límite de palabra
actual  secuencia literal
\b    límite de palabra
```

Este patrón no dice «busca cualquier cosa parecida a actual». Dice: busca exactamente la palabra actual, delimitada por fronteras de palabra.

2.4. Límites de palabra: evitar capturas falsas

El símbolo `\b` indica un límite de palabra. Sirve para marcar una frontera entre caracteres de palabra y caracteres que no son de palabra.

Esto es fundamental cuando se buscan términos léxicos. Si buscamos:

```
"actual"
```

podríamos encontrar esa secuencia dentro de otras palabras. En cambio, con:

```
r"\bactual\b"
```

se busca la palabra completa.

La diferencia es lingüísticamente relevante:

Forma	Debe coincidir con <code>\bactual\b</code>	Motivo
actual	Sí	Es la palabra completa
Actual	Sí, si usamos IGNORECASE	Es la misma palabra con mayúscula
actually	No	Contiene más letras después
factually	No	actual está dentro de otra palabra

Este detalle evita ruido. En detección de falsos amigos, no queremos que `actual` se confunda con `actually`. Son problemas distintos y requieren equivalencias distintas.

2.5. Buscar todas las coincidencias con `re.findall()`

`re.findall()` busca todas las apariciones de un patrón dentro de un texto y devuelve una lista.

```
patron = r"\bactual\b"
coincidencias = re.findall(patron, texto, flags=re.IGNORECASE)
```

Si el patrón aparece tres veces, la lista tendrá tres elementos. Por eso se puede contar así:

```
len(coincidencias)
```

La opción `flags=re.IGNORECASE` hace que la búsqueda ignore mayúsculas y minúsculas. Así, `Actual`, `ACTUAL` y `actual` se detectan como la misma forma.

En un texto bilingüe de prueba, el recuento puede producir resultados como:

Término buscado	Ocurrencias
actual	3
actually	1
sensible	2
eventually	2
assist	1
library	0

Este tipo de tabla responde a una primera pregunta cuantitativa: qué términos sospechosos aparecen y con qué frecuencia.

Pero el recuento no basta. Saber que una palabra aparece no dice todavía si hay un error. Para revisar, necesitamos contexto.

2.6. Localizar segmentos con `split('\n')`

Si el texto contiene varios segmentos separados por saltos de línea, se puede dividir así:

```
oraciones = texto.split('\n')
```

El resultado es una lista. Cada elemento corresponde a una línea del texto original.

Por ejemplo:

```
[
    "The patient is actually very sensible to the drug. El paciente es actualmente muy sensible al medicamento.",
    "Eventually, the physician decided to assist the patient. Eventualmente, el médico decidió asistir al paciente.",
    ...
]
```

Esta estrategia funciona porque cada línea representa un segmento bilingüe. En lugar de revisar todo el bloque como una unidad, podemos analizar segmento por segmento.

En PLN, segmentar correctamente es una decisión importante. Aquí no usamos `.split('.')` porque un punto aparece dentro de cada segmento y separaría de forma demasiado agresiva. Usamos `.split('\n')` porque el salto de línea marca la unidad de revisión que nos interesa.

2.7. Limpiar líneas con `strip()`

Cuando un texto multilínea empieza o termina con saltos de línea, pueden aparecer elementos vacíos en la lista. Por eso se usa:

```
oracion = oracion.strip()
```

`.strip()` elimina espacios, tabulaciones y saltos de línea al principio y al final de una cadena.

Después se puede comprobar:

```
if oracion:
    ...
```

En Python, una cadena vacía se interpreta como `False`; una cadena con contenido se interpreta como `True`. Por tanto, `if oracion:` significa: "solo continúa si esta línea tiene texto".

Esta limpieza evita que el programa intente analizar líneas vacías y genere salidas innecesarias.

2.8. Comprobar presencia con `re.search()`

`re.search()` busca un patrón dentro de una cadena y se detiene cuando encuentra la primera coincidencia. Si encuentra algo, devuelve un objeto de coincidencia. Si no encuentra nada, devuelve `None`.

Por eso se usa dentro de una condición:

```
if re.search(patron, oracion, flags=re.IGNORECASE):
    print(oracion)
```

La lectura es:

Si esta línea contiene el patrón, muéstrala.

La diferencia con `findall()` es fundamental:

Función	Qué devuelve	Para qué se usa
<code>re.findall()</code>	Lista de todas las coincidencias	Contar ocurrencias
<code>re.search()</code>	Primera coincidencia o None	Decidir si una línea se incluye en el informe

En control de calidad, `findall()` ayuda a medir. `search()` ayuda a filtrar.

2.9. Glosarios como diccionarios

Un diccionario de Python guarda pares clave-valor. Para detectar falsos amigos, podemos usar un diccionario donde:

- La clave sea la forma inglesa que queremos vigilar.
- El valor sea la traducción literal problemática en español.

Ejemplo:

```
falsos_amigos = {
    "actual": "actual",
    "actually": "actualmente",
    "sensible": "sensible",
    "eventually": "eventualmente",
    "assist": "asistir",
    "library": "librería"
}
```

Este diccionario representa conocimiento lingüístico contrastivo. No es una lista cualquiera. Es una hipótesis de revisión: «si aparece esta forma inglesa y además aparece esta forma española, puede haberse producido una transferencia errónea».

Conviene distinguir entre dos tipos de glosario:

Tipo de glosario	Qué almacena	Ejemplo
Glosario de riesgo	Traducción literal problemática	actually -> actualmente
Glosario de corrección	Equivalencia recomendada	actually -> en realidad

El primer glosario ayuda a detectar. El segundo ayuda a proponer.

2.10. Recorrer el glosario con un bucle for

Si hay seis falsos amigos, podríamos escribir seis búsquedas manuales. Pero eso sería poco eficiente. Un bucle permite repetir la misma operación para cada entrada del glosario.

```
for palabra in falsos_amigos:
    resultados = re.findall(fr"\b{palabra}\b", texto, flags=re.IGNORECASE)
    print(palabra, len(resultados))
```

Cuando se recorre un diccionario directamente, Python recorre sus claves. En este ejemplo, palabra irá tomando valores como:

```
"actual"
"actually"
"sensible"
"eventually"
"assist"
"library"
```

La ventaja es que el código se vuelve escalable. Si se añaden nuevos falsos amigos al diccionario, el bucle no cambia.

Desde el punto de vista profesional, esta es la diferencia entre un truco puntual y una herramienta reutilizable.

2.11. Patrones dinámicos con f-strings raw

Dentro del bucle necesitamos construir un patrón diferente para cada palabra. Para eso se usa:

```
fr"\b{palabra}\b"
```

Este string combina dos propiedades:

Prefijo	Función
f	Permite insertar el valor de una variable
r	Mantiene las barras invertidas como literales

Si palabra vale actually, el patrón final será:

```
\bactually\b
```

Si palabra vale library, el patrón final será:

```
\blibrary\b
```

Esta técnica es muy potente porque permite convertir un glosario en una serie automática de búsquedas.

2.12. Primer tipo de informe: segmentos sospechosos

El primer informe útil consiste en mostrar todos los segmentos donde aparece una palabra del glosario.

La lógica es:

1. Dividir el texto en líneas.
2. Recorrer cada línea.
3. Recorrer cada palabra del glosario.
4. Comprobar si esa palabra aparece en la línea.
5. Mostrar una advertencia.

Un esquema de código sería:

```
oraciones = texto.split('\n')

for oracion in oraciones:
    if oracion.strip():
        for palabra in falsos_amigos:
            if re.search(fr"\b{palabra}\b", oracion, flags=re.IGNORECASE):
                print("Segmento con posible falso amigo:", palabra)
                print(oracion)
```

Este informe detecta riesgos. Es útil para una primera criba, pero todavía puede generar falsos positivos.

Por ejemplo:

```
Eventually, the patient decided to look for a second opinion. Al final, el paciente
decidió buscar una segunda opinión.
```

El segmento contiene *Eventually*, pero la traducción *Al final* es correcta. Si el informe solo detecta la palabra inglesa, marcará un segmento que no contiene error.

Por eso se necesita una regla más precisa.

2.13. Segundo tipo de informe: error probable en TO y TM

Para reducir falsos positivos, se puede exigir una condición doble:

1. Que aparezca la palabra inglesa sospechosa.
2. Que aparezca también la traducción literal problemática en español.

En Python se expresa con `and`:

```
if (
    re.search(fr"\b{palabra}\b", oracion, flags=re.IGNORECASE)
    and re.search(fr"\b{traduccion_erronea}\b", oracion, flags=re.IGNORECASE)
):
    print(oracion)
```

Esta condición dice:

```
Muestra el segmento solo si contiene la forma inglesa y la forma española problemática.
```

Por ejemplo:

```
Actually, I wanted to visit the library. Actualmente, quería visitar la librería.
```

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

Este segmento cumple dos relaciones problemáticas:

Inglés	Español detectado	Motivo
Actually	Actualmente	Falso amigo
library	librería	Falso amigo

En cambio:

The nurse was eventually informed about the side effects. La enfermera fue informada finalmente sobre los efectos secundarios.

contiene eventually, pero no contiene eventualmente. Por tanto, no debería aparecer en el informe de errores probables.

Esta diferencia es esencial: no todo riesgo detectado es un error cometido.

2.14. Glosario de corrección: no solo avisar, también orientar

Un informe profesional debe intentar ser útil para la persona revisora. No basta con decir «aquí hay algo». Conviene indicar qué se ha detectado y qué alternativa podría valorarse.

Para eso se define un segundo diccionario:

```
falsos_amigos_corr = {
    "actual": "real",
    "actually": "en realidad, realmente",
    "sensible": "sensato",
    "eventually": "al final, finalmente",
    "assist": "ayudar",
    "library": "biblioteca"
}
```

Con los dos diccionarios, el informe puede mostrar:

Falso amigo detectado: actually ↔ actualmente
 Traducción correcta sugerida: en realidad, realmente

El símbolo ↔ no tiene valor computacional especial en este caso; se usa para presentar visualmente la relación entre la forma origen y la forma problemática en la traducción.

El sistema no impone una corrección única. Propone una equivalencia posible. En traducción, la solución depende del contexto, del género textual y de las convenciones terminológicas.

2.15. Qué ocurre con variantes morfológicas

Un patrón como:

```
r"\bassist\b"
```

detecta assist, pero no detecta necesariamente:

```
assisted
assisting
assistance
```

Esto es coherente con el patrón: hemos pedido una palabra exacta. Si queremos detectar una familia morfológica, necesitamos otra estrategia.

Una opción básica sería ampliar el patrón:

```
r"\bassist(ed|ing|ance)?\b"
```

Pero esta solución ya introduce más complejidad. Además, no todas las familias léxicas se comportan de forma tan sencilla.

Otra opción profesional sería usar herramientas de análisis morfológico o lematización. Un lematizador intenta reducir formas flexionadas a una forma base. Así, *assisted* podría relacionarse con el lema *assist*.

Esta limitación muestra una frontera importante: las expresiones regulares son muy útiles para patrones superficiales, pero no sustituyen un análisis lingüístico más profundo.

2.16. Falsos positivos y falsos negativos

En detección automática conviene distinguir dos tipos de error del sistema:

Tipo	Qué ocurre	Ejemplo
Falso positivo	El sistema marca un caso que no es error	eventually traducido correctamente como finalmente
Falso negativo	El sistema no marca un error real	assisted no detectado por buscar solo assist

Un buen sistema intenta reducir ambos, pero siempre hay una tensión:

- Si el patrón es muy amplio, detecta más casos, pero genera más ruido.
- Si el patrón es muy estricto, genera menos ruido, pero puede dejar escapar errores.

En control de calidad lingüística, esta tensión se parece a la relación entre cobertura y precisión:

- **Cobertura:** cuántos errores potenciales alcanza a detectar el sistema.
- **Precisión:** cuántos de los casos detectados son realmente relevantes.

La condición doble TO-TM mejora la precisión, porque exige más evidencia antes de mostrar un segmento. Pero puede reducir la cobertura si el error aparece con una variante no prevista en el glosario.

Errores frecuentes de aprendizaje

Confundir coincidencia con error: Que aparezca una palabra sospechosa no significa automáticamente que la traducción sea incorrecta.

Olvidar ``\b``: Sin límites de palabra, se pueden capturar fragmentos dentro de palabras más largas.

Olvidar ``re.IGNORECASE``: Si no se ignoran mayúsculas, el sistema puede no detectar términos al inicio de oración.

Confundir ``findall()`` y ``search()``: `findall()`` sirve para contar. `search()`` sirve para filtrar.

No revisar variantes flexionadas: Buscar `assist` no basta para detectar `assisted`.

Usar variables antiguas en entornos interactivos: En un entorno interactivo, una variable definida antes puede seguir activa. Eso puede ocultar errores si se reutilizan nombres sin cuidado.

Diseñar glosarios sin contexto: Un par de palabras no siempre basta. Muchas decisiones dependen del dominio y de la frase.

Síntesis final

En esta unidad has visto cómo una tarea lingüística puede convertirse en un procedimiento computacional. El punto de partida es un problema real: detectar falsos amigos en traducción automática biomédica. El punto de llegada es una primera herramienta de control de calidad capaz de buscar patrones, contar ocurrencias, localizar segmentos y proponer alternativas.

La idea central es que Python no «sabe» traducción. Lo que hace Python es ejecutar reglas. La parte lingüística consiste en decidir qué reglas merece la pena formular: qué términos son peligrosos, qué traducciones son sospechosas, qué contexto hay que mostrar y qué nivel de evidencia necesitamos antes de marcar un segmento.

Las expresiones regulares ofrecen una entrada muy poderosa al PLN aplicado porque permiten convertir observaciones lingüísticas en patrones verificables. Aun así, su potencia depende de la precisión del análisis previo. Una buena regex nace de una buena pregunta lingüística.

En el escenario de la empresa biomédica, esta herramienta no entrega una traducción final. Entrega algo igual de importante en un flujo profesional: una lista priorizada de lugares donde mirar. Esa lista ahorra tiempo, mejora la coherencia y añade una capa de seguridad lingüística antes de que el texto llegue al cliente o al usuario final.

Cierre narrativo

La empresa tecnológica no necesita solo más datos ni más modelos. Necesita mejores mecanismos de control. En traducción automática biomédica, la fluidez no basta: hay que vigilar la precisión léxica, la adecuación terminológica y la seguridad comunicativa.

Tu papel como lingüista computacional consiste en tender un puente entre conocimiento experto y automatización. Sabes qué errores son peligrosos, sabes cómo se manifiestan en los textos y aprendes a convertir ese conocimiento en reglas ejecutables.

Esa es la competencia de fondo de esta unidad: diseñar pequeñas herramientas que hagan visible lo que, en un corpus grande, podría quedar oculto. Python busca; la lingüista decide qué buscar, por qué importa y cómo debe interpretarse el resultado.

Referencias

- Bird, S., Klein, E. y Loper, E. (2009). Natural language processing with Python. O'Reilly Media. <https://www.nltk.org/book/>
- Chamizo Domínguez, P. J. (2008). Semantics and pragmatics of false friends. Routledge.
- Friedl, J. E. F. (2006). Mastering regular expressions (3rd ed.). O'Reilly Media. <https://www.oreilly.com/library/view/mastering-regular-expressions/0596528124/>
- Goyvaerts, J. y Levithan, S. (2009). Regular expressions cookbook. O'Reilly Media.
- Jurafsky, D. y Martin, J. H. (2026). Speech and language processing (3rd ed. draft). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- Kleene, S. C. (1956). Representation of events in nerve nets and finite automata. In C. E. Shannon & J. McCarthy (Eds.), Automata studies (pp. 3-42). Princeton University Press.
- Python Software Foundation. (2026). re: Regular expression operations. Python 3.14 documentation. <https://docs.python.org/3/library/re.html>